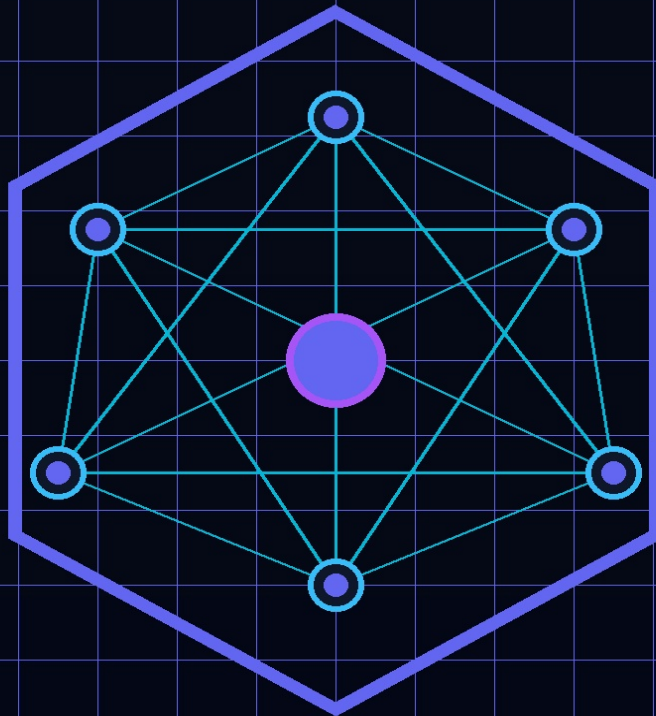


GUÍA DEFINITIVA DE SISTEMAS DISTRIBUIDOS

HEXAGEN ECOSYSTEM

MICROSERVICIOS



Event-Driven Architecture, CQRS a Escala
y Consistencia Eventual con Slices Distribuidos

RABBITMQ • APACHE KAFKA • GRPC • SAGAS • ZERO-TRUST

POR

Raul Alejandro Baez Camarillo

HexaGen: Microservicios y Slices Distribuidos

Event-Driven Architecture, CQRS a Escala y Consistencia Eventual

Por Raul Alejandro Baez Camarillo

Sinopsis de la Obra

La transición de un monolito a una arquitectura distribuida es uno de los mayores desafíos de ingeniería de software contemporánea. La mayoría de los proyectos fracasan al migrar prematuramente o al crear "monolitos distribuidos": sistemas acoplados por red con alta latencia, inconsistencia de datos y pesadillas operativas.

HexaGen: Microservicios y Slices Distribuidos ofrece una hoja de ruta pragmática y de nivel de producción para descomponer sistemas basados en **Vertical Slice Architecture (VSA)** en servicios autónomos, desacoplados por eventos y preparados para procesar millones de transacciones por segundo.

¿A quién está dirigido este libro?

- **Arquitectos de Software y Tech Leads:** Que necesitan diseñar sistemas distribuidos resilientes, escalables y auditables.
 - **Ingenieros Backend Senior:** Que buscan dominar Event-Driven Architecture (EDA), CQRS, el patrón Transactional Outbox, Sagas y Event Sourcing.
 - **Desarrolladores de Ecosistema HexaGen:** Que desean extender sus conocimientos de Vertical Slices hacia microservicios distribuidos sin perder la simplicidad arquitectónica.
-

Pila Tecnológica & Patrones

- **Lenguajes & Runtimes:** PHP 8.3+, FrankenPHP, Go (Servicios auxiliares de alta concurrencia).
 - **Mensajería & Streaming:** RabbitMQ (AMQP), Apache Kafka, Redis Streams.
 - **Protocolos:** gRPC / Protocol Buffers, HTTP/3, Server-Sent Events (SSE).
 - **Patrones Clave:** Vertical Slices Distribuidos, CQRS, Transactional Outbox, Sagas (Orquestadas y Coreografiadas), Event Sourcing, Circuit Breaker, mTLS Zero-Trust.
-

© 2026 Raul Alejandro Baez Camarillo • VortexSolutions • Hexagen Ecosystem. Todos los derechos reservados.

Capítulo 1: De la Vertical Slice Monolítica a los Slices Distribuidos

La evolución natural de una aplicación exitosa construida sobre **Vertical Slice Architecture (VSA)** suele enfrentarse a una encrucijada crítica: ¿cuándo es el momento adecuado para extraer slices hacia microservicios autónomos y cómo hacerlo sin caer en la trampa del monolito distribuido?

1. Los Tres Motores de la Descomposición

Descomponer un monolito no debe obedecer a modas tecnológicas, sino a necesidades operativas y de negocio cuantificables:

- **Escalabilidad Asimétrica:** Un caso de uso específico (como el procesamiento de webhooks de pagos o la ingesta de telemetría) demanda 50 veces más recursos de CPU y memoria que el resto del sistema.
 - **Ciclos de Despliegue y Autonomía de Equipos (Ley de Conway):** Equipos de ingeniería multidisciplinarios necesitan desplegar funcionalidades a producción múltiples veces al día sin riesgo de bloquear a otros equipos.
 - **Aislamiento de Fallos Críticos:** Un error fatal o fuga de memoria en un módulo no transaccional (ej. generación de reportes PDF pesados) no debe comprometer el motor transaccional de órdenes ni la pasarela de pagos.
-

2. Mapa de Transición Arquitectónica

Nivel Arquitectónico	Monolito Vertical Slice	Microservicios Distribuidos
Organización de Código	Directorios autocontenidos por slice (<code>app/Slices/...</code>)	Repositorios o servicios autónomos por Bounded Context
Persistencia	Esquema común con tablas separadas por slice	Base de datos dedicada y aislada por servicio (<i>Database-per-Service</i>)
Comunicación	Llamadas a Handlers en memoria	Eventos de integración asíncronos y contratos gRPC
Fronteras Transaccionales	Transacciones ACID locales	Consistencia Eventual y Patrón Saga con compensación

3. Delimitación de Contextos (Bounded Contexts)

En HexaGenPHP, cada Vertical Slice ya posee un alto nivel de cohesión y bajo acoplamiento. Al transicionar a microservicios, el primer paso consiste en agrupar slices afines dentro de un **Bounded Context** de Domain-Driven Design (DDD).

Reglas de Oro de los Slices Distribuidos:

1. **Base de Datos por Servicio (Database-per-Service):** Ningún servicio accede directamente a las tablas o esquemas de otro servicio. La comunicación ocurre estrictamente mediante APIs o eventos.
2. **Sin Compartir Modelos de Dominio:** Los DTOs de integración son contratos inmutables; no se comparten entidades de persistencia (Doctrine o Eloquent) entre repositorios.
3. **Desacoplamiento Temporal:** Un servicio debe poder procesar peticiones aunque sus dependencias aguas abajo se encuentren temporalmente fuera de línea.

Capítulo 2: Anatomía de un Microservicio HexaGen

Un microservicio dentro del ecosistema HexaGen mantiene la filosofía de puertos y adaptadores limpios combinada con Vertical Slice Architecture, pero incorpora interfaces de transporte distribuido.

1. Estructura de Directorios de un Servicio Distribuido

```
order-service/
├── src/
│   ├── Domain/                                # Reglas de negocio puras
│   │   ├── Model/Order.php
│   │   ├── ValueObjects/Money.php
│   │   └── Ports/
│   │       ├── OrderRepositoryPort.php
│   │       └── MessagePublisherPort.php
│   ├── Slices/                                # Casos de uso (VSA)
│   │   ├── CreateOrder/
│   │   │   ├── CreateOrderCommand.php
│   │   │   ├── CreateOrderHandler.php
│   │   │   └── CreateOrderEndpoint.php
│   │   └── CancelOrder/
│   ├── Infrastructure/                         # Adaptadores de tecnologia
│   │   ├── Persistence/DoctrineOrderRepository.php
│   │   ├── Messaging/RabbitMQMessagePublisher.php
│   │   └── Grpc/OrderGrpcServer.php
│   └── Shared/
│       └── Contracts/Events/OrderCreatedIntegrationEvent.php
```

2. Implementación de Puertos de Integración

```
namespace OrderService\Domain\Ports;

use OrderService\Shared\Contracts\Events\IntegrationEventInterface;

interface MessagePublisherPort
{
    public function publish(IntegrationEventInterface $event): void;
    public function publishBatch(array $events): void;
}
```

El dominio de la orden genera eventos de negocio puros, los cuales son traducidos a eventos de integración serializados antes de abandonar el proceso:

```
namespace OrderService\Slices\CreateOrder;

use OrderService\Domain\Model\Order;
use OrderService\Domain\Ports\OrderRepositoryPort;
use OrderService\Domain\Ports\MessagePublisherPort;
use OrderService\Shared\Contracts\Events\OrderCreatedIntegrationEvent;

final readonly class CreateOrderHandler
{
    public function __construct(
        private OrderRepositoryPort $repository,
        private MessagePublisherPort $publisher
    ) {}

    public function handle(CreateOrderCommand $cmd): string
    {
        $order = Order::create($cmd->customerId, $cmd->items);
        $this->repository->save($order);

        // Publicar evento de integración para otros servicios
        $this->publisher->publish(new OrderCreatedIntegrationEvent(
            orderId: $order->id(),
            customerId: $order->customerId(),
            totalAmount: $order->total()->amount(),
            currency: $order->total()->currency(),
            occurredOn: new \DateTimeImmutable()
        ));

        return $order->id();
    }
}
```

Capítulo 4: CQRS a Escala y Modelos de Lectura Desacoplados

La separación de responsabilidades de consulta y comando (**Command Query Responsibility Segregation**) permite optimizar los modelos de escritura para la consistencia transaccional y los modelos de lectura para la velocidad extrema.

1. El Desafío de las Consultas Distribuidas

En un monolito tradicional, generar una vista consolidada con datos de clientes, órdenes y pagos requiere un simple `JOIN` SQL. En una arquitectura de microservicios, intentar realizar múltiples llamadas HTTP síncronas entre servicios en cada petición de usuario genera:

- **Latencia multiplicativa:** Si 4 servicios tardan 50 ms cada uno, la consulta acumula más de 200 ms.
 - **Puntos únicos de fallo:** Si uno de los servicios dependientes cae, toda la consulta falla.
-

2. Flujo del Patrón de Proyecciones Asíncronas

1. **Lado de Escritura (Command):** El servicio procesa la acción de negocio y guarda los cambios en su base de datos transaccional (ej. PostgreSQL).
 2. **Emisión de Evento:** El servicio publica el evento de integración `OrderCreated` en el broker de mensajería.
 3. **Procesador de Proyección:** Un worker en segundo plano consume el evento y actualiza la vista desnormalizada en un almacén optimizado para lectura (ej. Elasticsearch o Redis).
 4. **Lado de Lectura (Query):** El API de consultas sirve la información en menos de 5 milisegundos directamente desde la proyección, sin consultar la base de datos de escritura.
-

3. Implementación de una Proyección Materializada

```
namespace ReportingService\Projections;

use ReportingService\Infrastructure\ElasticsearchClient;
use OrderService\Shared\Contracts\Events\OrderCreatedIntegrationEvent;

final readonly class OrderSummaryProjector
{
    public function __construct(private ElasticsearchClient $elastic) {}

    public function handle(OrderCreatedIntegrationEvent $event): void
    {
        $document = [
            'order_id' => $event->orderId,
            'customer_id' => $event->customerId,
            'total_amount' => $event->totalAmount,
            'status' => 'PENDING_PAYMENT',
            'created_at' => $event->occurredOn->format(\DateTimeInterface::ATOM)
        ];

        $this->elastic->index('orders_view', $event->orderId, $document);
    }
}
```


Capítulo 5: Consistencia Eventual y Patrón Transactional Outbox

El mayor riesgo en arquitecturas distribuidas es el problema de la escritura dual (*dual-write problem*): ¿qué ocurre si la base de datos guarda la orden, pero el servidor falla antes de enviar el evento a RabbitMQ? O peor: ¿si el mensaje se envía pero la transacción de base de datos hace Rollback?

1. El Patrón Transactional Outbox

Para garantizar entrega de mensajes sin bloqueos distribuidos (Two-Phase Commit / 2PC), los eventos se guardan dentro de la **misma transacción atómica SQL** que los datos de negocio en una tabla especial denominada `outbox_events`.

Flujo de Ejecución en 2 Fases:

- **Fase 1 (Transacción Local Atómica):**
 - `INSERT INTO orders (...) VALUES (...);`
 - `INSERT INTO outbox_events (event_type, payload, status) VALUES (...);`
 - `COMMIT;` (Ambas operaciones tienen éxito o ambas fallan).
 - **Fase 2 (Relay Asíncrono):**
 - Un proceso worker en segundo plano lee los eventos pendientes, los publica en el Message Broker y actualiza el estado a `PROCESSED`.
-

2. Esquema SQL del Outbox

```
CREATE TABLE outbox_events (  
  id CHAR(36) PRIMARY KEY,  
  event_type VARCHAR(255) NOT NULL,  
  aggregate_id VARCHAR(64) NOT NULL,  
  payload JSON NOT NULL,  
  status ENUM('PENDING', 'PROCESSED', 'FAILED') DEFAULT 'PENDING',  
  retry_count INT DEFAULT 0,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  processed_at TIMESTAMP NULL,  
  INDEX idx_status_created (status, created_at)  
);
```

3. Implementación del Dispatcher de Outbox

```
namespace HexaGen\Distributed\Outbox;

use PDO;

final readonly class OutboxRelay
{
    public function __construct(
        private PDO $db,
        private MessagePublisherInterface $publisher
    ) {}

    public function processBatch(int $limit = 100): int
    {
        $stmt = $this->db->prepare("
            SELECT id, event_type, payload
            FROM outbox_events
            WHERE status = 'PENDING'
            ORDER BY created_at ASC
            LIMIT :limit FOR UPDATE SKIP LOCKED
        ");
        $stmt->bindValue(':limit', $limit, PDO::PARAM_INT);
        $stmt->execute();
        $events = $stmt->fetchAll(PDO::FETCH_ASSOC);

        foreach ($events as $event) {
            $this->publisher->publish($event['event_type'], $event['payload']);

            $update = $this->db->prepare("
                UPDATE outbox_events
                SET status = 'PROCESSED', processed_at = NOW()
                WHERE id = :id
            ");
            $update->execute([':id' => $event['id']]);
        }

        return count($events);
    }
}
```

Capítulo 6: Transacciones Distribuidas: El Patrón Saga

Cuando un proceso de negocio involucra múltiples microservicios (por ejemplo: Checkout de Orden → Cobro de Tarjeta → Reserva de Inventario → Envío de Factura), no es viable utilizar una transacción ACID tradicional. El **Patrón Saga** resuelve esto mediante una serie de transacciones locales coordinadas y **acciones compensatorias**.

1. Coreografía vs. Orquestación

Aspecto	Saga Coreografiada	Saga Orquestada
Control	Descentralizado; cada servicio reacciona a eventos de otros	Centralizado; un Orquestador dirige cada paso
Acoplamiento	Muy bajo a nivel de infraestructura	Ligeramente mayor hacia el orquestador
Complejidad de Trazabilidad	Difícil de rastrear cuando hay más de 5 pasos	Clara y explícita en una máquina de estados
Recomendado para	Flujos simples de 2 a 3 servicios	Flujos transaccionales complejos o financieros

2. Máquina de Estados de una Saga Orquestada

```
namespace OrderService\Sagas\Checkout;

final class CheckoutSagaOrchestrator
{
    public function __construct(
        private PaymentClientPort $payments,
        private InventoryClientPort $inventory,
        private NotificationClientPort $notifications
    ) {}

    public function execute(CheckoutSagaData $data): void
    {
        // Paso 1: Intentar autorización de cobro
        $paymentResult = $this->payments->authorizePayment($data->orderId, $data->amount);
        if (!$paymentResult->isSuccess()) {
            $this->compensate($data, 'PAYMENT_FAILED');
            return;
        }

        // Paso 2: Intentar reserva de stock en almacén
        $stockResult = $this->inventory->reserveItems($data->orderId, $data->items);
        if (!$stockResult->isSuccess()) {
            // ACCIÓN COMPENSATORIA: Reembolsar dinero cobrado previamente
            $this->payments->refundPayment($data->orderId, $data->amount);
            $this->compensate($data, 'INSUFFICIENT_STOCK');
            return;
        }

        // Paso 3: Notificar confirmación exitosa
        $this->notifications->sendConfirmation($data->customerId, $data->orderId);
    }

    private function compensate(CheckoutSagaData $data, string $reason): void
    {
        // Marcar orden como rechazada y registrar auditoría forense
    }
}
```

Capítulo 7: Comunicación Síncrona vs Asíncrona: gRPC y REST

Aunque los eventos asíncronos son el pilar de la escalabilidad, existen escenarios donde se requiere una respuesta síncrona inmediata de ultra baja latencia (ej. verificación de identidad en milisegundos o validaciones de riesgo en tiempo real).

1. Por qué gRPC con Protocol Buffers

- **Eficiencia Binaria:** Protocol Buffers reduce el tamaño del payload hasta en un 70% comparado con JSON.
- **HTTP/2 Multiplexing:** Múltiples peticiones concurrentes viajan sobre una única conexión TCP persistente.
- **Contratos Fuertemente Tipados:** Los archivos `.proto` son la única fuente de verdad para generar clientes y servidores en PHP, Go, Rust o Node.js.

```
syntax = "proto3";

package hexagen.payments.v1;

service PaymentService {
  rpc ProcessPayment (PaymentRequest) returns (PaymentResponse);
}

message PaymentRequest {
  string transaction_id = 1;
  string customer_id = 2;
  double amount = 3;
  string currency = 4;
}

message PaymentResponse {
  string status = 1;
  string authorization_code = 2;
  int64 processed_at_timestamp = 3;
}
```

2. Consumo de Servicios gRPC en PHP 8.3

```
namespace App\Infrastructure\Adapters;

use Hexagen\Payments\V1\PaymentServiceClient;
use Hexagen\Payments\V1\PaymentRequest;
use App\Domain\Ports\PaymentGatewayPort;

final readonly class GrpcPaymentGatewayAdapter implements PaymentGatewayPort
{
    private PaymentServiceClient $client;

    public function __construct(string $hostname)
    {
        $this->client = new PaymentServiceClient($hostname, [
            'credentials' => \Grpc\ChannelCredentials::createInsecure(),
        ]);
    }

    public function charge(string $txId, string $customer, float $amount, string $currency): bool
    {
        $request = new PaymentRequest();
        $request->setTransactionId($txId);
        $request->setCustomerId($customer);
        $request->setAmount($amount);
        $request->setCurrency($currency);

        [$response, $status] = $this->client->ProcessPayment($request)->wait();

        if ($status->code !== \Grpc\STATUS_OK) {
            throw new \RuntimeException("gRPC Payment Call Failed: " . $status->details);
        }

        return $response->getStatus() === 'APPROVED';
    }
}
```

Capítulo 8: Event Sourcing: El Estado como Secuencia Inmutable

En aplicaciones financieras, de auditoría o de logística crítica, sobrescribir registros con [UPDATE](#) destruye el rastro histórico del negocio. **Event Sourcing** modela el estado de una entidad como el resultado acumulado de una secuencia inmutable de eventos.

1. Comparativa: Modelo CRUD vs. Event Sourcing

Modelo Tradicional CRUD

- La tabla almacena únicamente la foto actual (`Balance: $750`).
- No existe registro histórico de cómo ni por qué se llegó a esa cifra sin recurrir a logs externos.

Modelo Event Sourcing

- El estado es el resultado de reproducir la secuencia de eventos:
 - `AccountOpened` (`InitialDeposit: $1,000.00`)
 - `MoneyWithdrawn` (`Amount: $300.00`)
 - `MoneyDeposited` (`Amount: $50.00`)
 - **Estado Reconstruido:** `$1,000 - $300 + $50 = $750.00`
 - **Beneficios:** Auditoría forense 100% matemática y capacidad de reproducir el estado del sistema en cualquier punto del tiempo (*Time-Travel Debugging*).
-

2. Reconstrucción de un Agregado desde sus Eventos

```
namespace HexaGen\EventSourcing;

abstract class AggregateRoot
{
    private int $version = 0;
    private array $recordedEvents = [];

    public function recordThat(object $event): void
    {
        $this->apply($event);
        $this->recordedEvents[] = $event;
    }

    public function reconstituteFromHistory(array $events): void
    {
        foreach ($events as $event) {
            $this->apply($event);
            $this->version++;
        }
    }

    abstract protected function apply(object $event): void;

    public function releaseEvents(): array
    {
        $events = $this->recordedEvents;
        $this->recordedEvents = [];
        return $events;
    }
}
```

Capítulo 9: Observabilidad Distribuida: OpenTelemetry y Tracing

En un sistema distribuido, rastrear una petición que atraviesa 6 microservicios distintos y 2 colas de mensajería es imposible sin observabilidad unificada basada en los tres pilares: **Logs estructurados**, **Métricas** y **Distributed Tracing**.

1. Propagación de Contexto W3C TraceContext

Cada petición que ingresa al sistema a través del API Gateway recibe un header estandarizado

`traceparent` :

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
|      |_____ Trace ID _____|  | Span ID |  |Flags
Version
```

Este identificador se propaga automáticamente a través de los clientes HTTP (Guzzle/Symfony HttpClient) y de los metadatos de los mensajes en RabbitMQ y Kafka, permitiendo visualizar la cascada completa de ejecución en herramientas como Jaeger o Grafana Tempo.

2. Instrumentación de OpenTelemetry en PHP 8.3

```
namespace HexaGen\Observability;

use OpenTelemetry\API\Globals;
use OpenTelemetry\API\Trace\TracerInterface;

final class TracingMiddleware
{
    private TracerInterface $tracer;

    public function __construct()
    {
        $this->tracer = Globals::tracerProvider()->getTracer('hexagen.order_service');
    }

    public function execute(string $operationName, callable $action)
    {
        $span = $this->tracer->spanBuilder($operationName)->startSpan();
        $scope = $span->activate();

        try {
            $result = $action();
            $span->setStatus(\OpenTelemetry\API\Trace\StatusCode::STATUS_OK);
            return $result;
        } catch (\Throwable $e) {
            $span->recordException($e);
            $span->setStatus(\OpenTelemetry\API\Trace\StatusCode::STATUS_ERROR, $e->getMessage());
            throw $e;
        } finally {
            $scope->detach();
            $span->end();
        }
    }
}
```


Capítulo 10: Resiliencia, Tolerancia a Fallos y Circuit Breakers

En sistemas distribuidos los fallos no son una excepción, son una certeza estadística: la red se satura, las bases de datos tienen picos de latencia y los servicios de terceros colapsan. Diseñar para el fallo es mandatorio.

1. Estados del Patrón Circuit Breaker

- **CLOSED (Cerrado - Normal):** Las peticiones fluyen directamente al servicio de destino. Se contabilizan fallos consecutivos.
 - **OPEN (Abierto - Protección):** Tras alcanzar el umbral de fallos (ej. 5 errores consecutivos), el circuito se abre inmediatamente. Todas las llamadas devuelven un *fallback* instantáneo sin saturar la red ni el servicio caído.
 - **HALF-OPEN (Semi-abierto - Prueba):** Tras un periodo de enfriamiento (ej. 30 segundos), se permite el paso de una petición de prueba. Si es exitosa, el circuito se restablece a **CLOSED**; si falla, vuelve a **OPEN**.
-

2. Implementación de Circuit Breaker con Redis en PHP 8.3

```
namespace HexaGen\Resilience;

final class CircuitBreaker
{
    private const STATE_CLOSED = 'CLOSED';
    private const STATE_OPEN = 'OPEN';
    private const STATE_HALF_OPEN = 'HALF_OPEN';

    public function __construct(
        private \Redis $redis,
        private string $serviceName,
        private int $failureThreshold = 5,
        private int $recoveryTimeoutSeconds = 30
    ) {}

    public function call(callable $action, callable $fallback)
    {
        $state = $this->getState();

        if ($state === self::STATE_OPEN) {
            return $fallback(); // Degeneración elegante sin saturar el servicio caído
        }

        try {
            $result = $action();
            $this->onSuccess();
            return $result;
        } catch (\Throwable $e) {
            $this->onFailure();
            return $fallback();
        }
    }

    private function getState(): string
    {
        return $this->redis->get("cb:{$this->serviceName}:state") ?: self::STATE_CLOSED;
    }

    private function onFailure(): void
    {
        $failures = $this->redis->incr("cb:{$this->serviceName}:failures");
        if ($failures >= $this->failureThreshold) {
            $this->redis->setex("cb:{$this->serviceName}:state", $this->recoveryTimeoutSeconds,
                self::STATE_OPEN);
        }
    }

    private function onSuccess(): void
    {
        $this->redis->del("cb:{$this->serviceName}:failures");
        $this->redis->set("cb:{$this->serviceName}:state", self::STATE_CLOSED);
    }
}
```

Capítulo 11: Seguridad Zero-Trust y Service Mesh en Microservicios

En un esquema de seguridad perimetral clásico, se asume que todo el tráfico dentro de la red interna es confiable. En **Zero-Trust**, ningún microservicio confía en otro de forma implícita: cada llamada es autenticada, cifrada y autorizada punto a punto.

1. Principios Zero-Trust en HexaGen

1. **Autenticación Mutua TLS (mTLS):** Todos los microservicios intercambian certificados X.509 verificados antes de transmitir un solo byte de información.
 2. **Tokens de Identidad Criptográficos (Ed25519 / JWTs Asimétricos):** Cada llamada inter-servicio lleva un token firmado que certifica el contexto del tenant y los roles del usuario.
 3. **Políticas de Red Estrictas (Network Policies):** Un servicio de pagos solo puede comunicarse con el servicio de órdenes y su base de datos aislada.
-

2. Verificación de Tokens Inter-Servicio en PHP 8.3

```
namespace HexaGen\Security;

use Firebase\JWT\JWT;
use Firebase\JWT\Key;

final class InterServiceTokenVerifier
{
    public function __construct(private string $publicKeyEd25519) {}

    public function verify(string $jwtToken): ServiceIdentity
    {
        $decoded = JWT::decode($jwtToken, new Key($this->publicKeyEd25519, 'EdDSA'));

        // Validar expiración y emisor autorizado
        if ($decoded->iss !== 'hexagen-auth-authority') {
            throw new \SecurityException("Emisor de token no autorizado");
        }

        return new ServiceIdentity(
            serviceId: $decoded->sub,
            tenantId: $decoded->tenant_id,
            allowedScopes: $decoded->scopes
        );
    }
}
```

Capítulo 12: Proyecto Completo de Producción: Sistema Transaccional de Pagos y Logística

Para consolidar todos los conceptos aprendidos, en este capítulo final ensamblaremos un sistema distribuido completo de grado de producción: **HexaLogistics Enterprise**.

1. Topología del Sistema

- **API Gateway (Envoy):** Punto de entrada único con terminación TLS, rate limiting y enrutamiento por path.
 - **Order Service (PHP 8.3 / VSA):** Gestión del ciclo de vida de órdenes y publicación Outbox hacia RabbitMQ.
 - **Payment Service (PHP 8.3 / VSA):** Autorización de cobros y orquestación de la Saga de checkout.
 - **Catalog Service (PHP 8.3):** Consultas de alta velocidad con caché distribuida en Redis.
 - **Notification Service (Go):** Worker concurrente para envío masivo de correos y webhooks.
-

2. Docker Compose de Producción

```
version: '3.8'

services:
  rabbitmq:
    image: rabbitmq:3.13-management-alpine
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: hexagen
      RABBITMQ_DEFAULT_PASS: hexagen_secret

  redis:
    image: redis:7.2-alpine
    ports:
      - "6379:6379"

  order-service:
    build:
      context: ./services/order-service
    environment:
      DATABASE_URL: mysql://hexagen:secret@order-db:3306/orders
      RABBITMQ_HOST: rabbitmq
    depends_on:
      - rabbitmq
      - redis

  payment-service:
    build:
      context: ./services/payment-service
    environment:
      DATABASE_URL: mysql://hexagen:secret@payment-db:3306/payments
      RABBITMQ_HOST: rabbitmq
    depends_on:
      - rabbitmq
```

3. Conclusión y Próximos Pasos en el Ecosistema

Has dominado la transición desde Vertical Slices monolíticos hacia un ecosistema distribuido resiliente, escalable y auditado con Event-Driven Architecture, CQRS y consistencia eventual.

El código limpio, los contratos inmutables y la disciplina arquitectónica son la clave para construir software de clase mundial con el **Hexagen Ecosystem**.